



UNIVERSITÀ DEGLI STUDI DI URBINO

DIPARTIMENTO DI SCIENZE PURE E APPLICATE

CORSO DI LAUREA IN INFORMATICA APPLICATA

Progetto di Programmazione Logica e Funzionale

di

Aboufaris Nasrine, Giuseppetti Alessia

Matricola: 321885, 322984

Anno di corso: terzo

Anno Accademico 2024/2025 - Sessione autunnale

Docente: Prof. Marco Bernardo

Indice

1	Specifica del problema	2
2	Analisi del problema	3
2.1	Dati in ingresso del problema	3
2.2	Dati in uscita del problema	3
2.3	Relazioni intercorrenti	3
2.3.1	Relazioni tra dati di ingresso	3
2.3.2	Relazione tra dati di ingresso e dati di uscita	3
3	Progettazione dell'algoritmo	5
3.1	Scelte di progetto	5
3.2	Passi dell'algoritmo	5
4	Implementazione dell'algoritmo	7
4.1	Implementazione in Haskell	7
4.2	Implementazione in Prolog	14
5	Testing	25
5.1	Testing del programma Haskell	25
5.2	Testing del programma Prolog	29

1 Specifica del problema

Scrivere un programma Haskell e un programma Prolog che acquisiscono dalla tastiera un messaggio binario, il quale verrà codificato o decodificato mediante un codice di Hamming generico (n,k) , in base alla scelta dell'utente. I parametri (n,k) saranno derivati da un valore m , fornito dall'utente, che rappresenta il numero di bit di parità. Il programma sarà inoltre in grado di calcolare la distanza di Hamming tra due stringhe binarie, fornite dall'utente, di lunghezza uguale ma arbitraria.

2 Analisi del problema

2.1 Dati in ingresso del problema

I dati in ingresso del problema sono:

- Una scelta tra le operazioni disponibili(codifica, decodifica o calcolo distanza);
 - Per la codifica: un numero naturale $m \geq 2$, che rappresenta il numero di bit di parità e una parola binaria di lunghezza $k = 2^m - m - 1$;
 - Per la decodifica: un numero naturale $m \geq 2$, che rappresenta il numero di bit di parità e una parola binaria di lunghezza $n = 2^m - 1$;
 - Per il calcolo della distanza: due parole binarie di uguale lunghezza $L \geq 1$.

2.2 Dati in uscita del problema

I dati in uscita del problema sono:

- Per la codifica: una parola binaria(composta da sequenze di bit) di lunghezza $n = 2^m - 1$, che rappresenta la parola codificata;
- Per la decodifica: una parola binaria di lunghezza $k = 2^m - m - 1$, che rappresenta la parola decodificata ed eventualmente corretta;
- Per il calcolo della distanza: un numero naturale $d \geq 0$, che rappresenta la distanza di Hamming.

2.3 Relazioni intercorrenti

2.3.1 Relazioni tra dati di ingresso

- Il valore m determina univocamente i parametri del codice: $n = 2^m - 1$ e $k = 2^m - m - 1$;
- La lunghezza delle parole in input deve essere coerente con l'operazione scelta e con il valore m ;
- Per il calcolo della distanza, le due parole devono avere la stessa lunghezza $L \geq 1$.

2.3.2 Relazione tra dati di ingresso e dati di uscita

- I bit di parità vengono inseriti nelle posizioni $p_i = 2^{(i-1)}$ per $i = 1, 2, \dots, m$, quindi posizioni potenze di due;
- La codifica trasforma una parola di lunghezza k in una parola di lunghezza n , aggiungendo m bit di parità;
- La decodifica utilizza i bit di parità per calcolare un vettore di sindrome:
 - Se $\text{sindrome} = 0$, non vengono rilevati errori;
 - Se $\text{sindrome} \neq 0$, individua la posizione di un eventuale errore singolo.

- La distanza di Hamming tra due parole è pari al numero di posizioni in cui i bit corrispondenti differiscono.

Inoltre è importante indicare determinate proprietà del codice di Hamming:

- Il codice di Hamming(n,k) ha distanza minima pari a 3, garantendo la correzione di errori singoli;
- I bit di parità nelle posizioni potenze di 2 permettono di identificare univocamente la posizione di un errore singolo tramite il vettore sindrome;
- La correzione è garantita solo per errori singoli, con errori multipli il comportamento non è definito.

3 Progettazione dell'algoritmo

3.1 Scelte di progetto

Le parole binarie verranno rappresentate mediante strutture dati lineari (liste) per preservare l'ordine dei bit, che è fondamentale per il corretto funzionamento del codice di Hamming. Le liste supportano naturalmente le operazioni bit a bit necessarie per i calcoli di parità e sindrome. I bit di parità verranno posizionati nelle posizioni che sono potenze di 2, mentre i bit informativi occuperanno le posizioni rimanenti. Questa disposizione garantisce la proprietà fondamentale del codice di Hamming per la correzione di errori singoli. La sindrome verrà calcolata come vettore binario che, quando diverso da zero, indica direttamente la posizione dell'errore nella parola codificata.

3.2 Passi dell'algoritmo

I passi dell'algoritmo per risolvere il problema sono i seguenti:

- Acquisire la scelta dell'operazione da eseguire tra codifica, decodifica e calcolo distanza di Hamming;
- Per la codifica:
 - Acquisire il parametro m (numero di bit di parità);
 - Calcolare $n = 2^m - 1$ e $k = n - m = 2^m - m - 1$;
 - Acquisire una parola binaria di lunghezza k ;
 - Posizionare i bit informativi nelle posizioni non potenze di 2;
 - Calcolare i bit di parità per ogni posizione potenza di 2:
 - * Caso base: se tutte le posizioni di parità sono state calcolate, terminare;
 - * Caso generale: per ogni bit di parità, calcolare XOR dei bit nelle posizioni controllate.
 - Resituire la parola codificata di lunghezza n .
- Per la decodifica:
 - Acquisire il parametro m e calcolare n e k ;
 - Acquisire una parola binaria di lunghezza n ;
 - Calcolare il vettore sindrome verificando tutti i controlli di parità:
 - * Caso base: tutti i controlli sono completati, la sindrome è determinata;
 - * Caso generale: per ogni controllo di parità fallito, impostare il bit corrispondente alla sindrome.
 - Se sindrome $\neq 0$, correggere il bit nella posizione indicata dalla sindrome;
 - Estrarre i bit informativi dalle posizioni non potenze di 2;
 - Resituire la parola decodificata di lunghezza k .
- Per la distanza di Hamming:
 - Acquisire due parole binarie di uguale lunghezza;

- Confrontare bit a bit le due parole:
 - * Caso base: se entrambe le parole sono vuote, la distanza è 0;
 - * Caso generale: confrontare il primo bit, incrementare la distanza se diversi, procedere ricorsivamente.
- Restituire la distanza come numero naturale.

4 Implementazione dell'algoritmo

L'I/O è stato gestito diversamente nei due programmi Haskell e Prolog. In Haskell, l'input viene tipicamente letto come stringa di caratteri, poiché l'I/O è basato su flussi di testo. Il programma convalida la stringa controllando che contenga solo '0' e '1' e che abbia lunghezza corretta.

In Prolog l'input viene letto direttamente come termine Prolog. Significa che l'input viene inserito come una lista nel formato sintattico di Prolog. Questo permette al programma di ottenere direttamente una struttura dati senza bisogno di conversione, sfruttando l'unificazione e il backtracking per la validazione.

4.1 Implementazione in Haskell

Per via dell'editor di testo usato e del linguaggio di marcatura ad esso associato, l'indentazione del codice qui presentato presenta minuscole differenze con quelli presenti nel file sorgente. Per avere la migliore indentazione e leggibilità, si prega di consultare il codice originale, contenuto dentro "CodiceHamming.hs".


```

-- #####
-- #      Corso di Programmazione Logica e Funzionale      #
-- #  Progetto per la sessione autunnale A.A. 2024/2025    #
-- #      di Masrine Aboufaris, Matricola: 321885          #
-- #      e Alessia Giuseppetti, Matricola: 322984          #
-- #      Anno di corso: Terzo                             #
-- #####

{- Specifica: Scrivere un programma Haskell che acquisisce da
    tastiera un messaggio binario,
    il quale verrà codificato o decodificato mediante un codice di
    Hamming generico (n,k), in base
    alla scelta dell'utente. I parametri (n,k) saranno derivati da un
    valore m, fornito dall'utente,
    che rappresenta il numero di bit di parità. Il programma sarà
    inoltre in grado di calcolare la
    distanza di Hamming tra due stringhe binarie, fornite dall'utente,
    di lunghezza uguale ma arbitraria.-}

import Data.Bits (testBit, xor, (.&.), shiftL)
import Text.Read (readMaybe)
import Control.Monad (unless, when)
import Data.List (foldl')
import System.Exit (exitSuccess)

-- Tipi di dati per migliorare la leggibilità del codice
type Bit = Int
type ParolaBinaria = [Bit]

{- main: Funzione principale del programma -}
main :: IO ()
main = do
    print
    "-----Benvenuto-----"
    print "Il seguente programma permette di utilizzare i codici di
        Hamming generici (n,k)"
    print "per la codifica, decodifica e calcolo della distanza di
        Hamming tra due parole"
    print "binarie."
    print "Come funziona il programma:"
    print "Potrai scegliere dal menu una delle tre operazioni da
        eseguire, per la codifica"
    print "e decodifica di Hamming, verra\' chiesto di inserire m,
        ovvero il numero di bit"
    print "di parita\' , grazie ad m verranno calcolati n e k, e si
        potra\' inserire la"
    print "parola binaria. Esempio di parola accetta '1011'"
    print "Notare che per la decodifica questo algoritmo corregge
        solo errori singoli."
    print "Con errori multipli il risultato potrebbe essere errato."
    print "Il programma finisce solo nel momento in cui si sceglie
        l'opzione esci dal menu"
    print
    "-----"
    cicloPrincipale

{- cicloPrincipale: Gestisce il menu principale e le scelte
    dell'utente -}

```

```

cicloPrincipale :: IO ()
cicloPrincipale = do
    -- Lista delle operazioni disponibili nel programma
    let vociMenu = [ "Codifica di Hamming"
                    , "Decodifica di Hamming"
                    , "Distanza di Hamming"
                    , "Esci"
                    ]

    -- Stampa il menu usando mapM_ per eseguire l'azione su ogni
    -- elemento
    putStrLn "\nScegli operazione:"
    mapM_ (\(indice, voce) -> putStrLn $ show indice ++ " - " ++
        voce)
        (zip [1..] vociMenu)

    -- Pattern matching sull'input dell'utente per determinare
    -- l'azione
    scelta <- getLine
    case scelta of
        "1" -> gestioneCodificaHamming >> cicloPrincipale
        "2" -> gestioneDecodificaHamming >> cicloPrincipale
        "3" -> gestioneDistanzaHamming >> cicloPrincipale
        "4" -> exitSuccess
        _    -> do
            putStrLn "Scelta non valida. Riprova."
            cicloPrincipale

{- gestioneCodificaHamming: Gestisce il processo di codifica di
Hamming
coordina input utente, calcoli e output per la codifica -}
gestioneCodificaHamming :: IO()
gestioneCodificaHamming = do
    putStrLn "\nCodifica di Hamming"
    parametroM <- controlloParametroM
    -- Calcola automaticamente n e k basandosi su m
    let (parametroN, parametroK) = calcolaParametriHamming parametroM
    putStrLn $ "Parametri calcolati: n = " ++ show parametroN ++ ",
        k = " ++ show parametroK
    parola <- controlloParolaBinaria parametroK
    {- Converte la stringa in lista di bit usando map e read -}
    let parolaDati = map (read . (:[])) parola :: ParolaBinaria
    let parolaCodificata = codificaHamming parametroM parolaDati
    putStrLn $ "La parola codificata è: " ++ concatMap show
        parolaCodificata

{- gestioneDecodificaHamming: Gestisce il processo di decodifica di
Hamming
coordina input utente, calcoli e output per la decodifica -}
gestioneDecodificaHamming :: IO()
gestioneDecodificaHamming = do
    putStrLn "\nDecodifica di Hamming"
    parametroM <- controlloParametroM
    let (parametroN, parametroK) = calcolaParametriHamming parametroM
    putStrLn $ "Parametri calcolati: n = " ++ show parametroN ++ ",
        k = " ++ show parametroK
    parola <- controlloParolaBinaria parametroN
    let parolaRicevuta = map (read . (:[])) parola :: ParolaBinaria

```

```

let (parolaDecodificata, errore) = decodificaHamming parametroM
    parolaRicevuta
-- Pattern matching su Maybe per gestire presenza o assenza di
  errore
case errore of
  Nothing ->
    putStrLn $ "Nessun errore rilevato. La parola
      decodificata è: " ++ concatMap show parolaDecodificata
  Just posizione ->
    putStrLn $ "Corretto un errore nella posizione " ++ show
      posizione ++
      ". La parola decodificata è: " ++ concatMap
        show parolaDecodificata

{- gestioneDistanzaHamming: Gestisce il calcolo della distanza di
  Hamming
  coordina input di due parole e calcolo della distanza -}
gestioneDistanzaHamming :: IO()
gestioneDistanzaHamming = do
  putStrLn "\nDistanza di Hamming"
  (s1, s2) <- controlloDueParole
  let p1 = map (read . (:[])) s1 :: ParolaBinaria
  let p2 = map (read . (:[])) s2 :: ParolaBinaria
  putStrLn $ "La distanza di Hamming tra le due parole è: " ++
    show (calcolaDistanzaHamming p1 p2)

{- controlloParametroM: Valida l'input del parametro m
  continua a chiedere input fino a ottenere un valore valido -}
controlloParametroM :: IO Int
controlloParametroM = do
  putStrLn "Inserisci m (m >= 2):"
  input <- getLine
  -- Usa readMaybe per parsing sicuro che ritorna Maybe Int
  case readMaybe input of
    Just m | m >= 2 -> return m
    _ -> do
      putStrLn "Errore: m deve essere un intero maggiore
        uguale a 2"
      controlloParametroM

{- controlloParolaBinaria: Valida l'input di una parola binaria di
  lunghezza specifica
  Argomenti: lunghezza richiesta della parola
  continua a chiedere input fino a ottenere una parola valida -}
controlloParolaBinaria :: Int -> IO String
controlloParolaBinaria lunghezza = do
  putStrLn $ "Inserisci parola binaria di lunghezza " ++ show
    lunghezza ++ ":"
  parola <- getLine
  -- all verifica che tutti i caratteri siano in "01"
  if all (elem "01") parola && length parola == lunghezza
    then return parola
    else do
      putStrLn "Errore: lunghezza non valida o caratteri non
        binari"
      controlloParolaBinaria lunghezza

```

```

{- controlloDueParole: Valida l'input di due parole binarie di
    uguale lunghezza
    continua a chiedere input fino a ottenere due parole valide -}
controlloDueParole :: IO (String, String)
controlloDueParole = do
    putStrLn "Inserisci prima parola binaria:"
    p1 <- getLine
    putStrLn "Inserisci seconda parola binaria (stessa lunghezza):"
    p2 <- getLine
    if length p1 == length p2 && all ('elem' "01") p1 && all ('elem'
        "01") p2
    then return (p1, p2)
    else do
        putStrLn "Errore: le parole devono essere binarie e di
            uguale lunghezza"
        controlloDueParole

{- calcolaParametriHamming: Calcola i parametri n e k del codice di
    Hamming
    Argomenti: m (numero di bit di parità)
    calcola  $n = 2^m - 1$  e  $k = n - m$  -}
calcolaParametriHamming :: Int -> (Int, Int)
calcolaParametriHamming m = (n, n - m)
    -- where permette di definire n localmente per riutilizzarlo
    where n = 2 ^ m - 1

{- potenzaDiDue: Verifica se un numero è una potenza di 2
    Argomenti: n (numero da verificare) -}
potenzaDiDue :: Int -> Bool
potenzaDiDue n = n /= 0 && (n .&. (n-1)) == 0

{- aggiornaElemento: Aggiorna un elemento in una lista a un indice
    specifico
    Argomenti: lista, indice, nuovo valore
    divide la lista e ricostruisce con nuovo valore -}
aggiornaElemento :: [a] -> Int -> a -> [a]
aggiornaElemento lista indice nuovo =
    -- splitAt divide la lista in due parti all'indice specificato
    let (prima, _:dopo) = splitAt indice lista
    in prima ++ [nuovo] ++ dopo

{- codificaHamming: Codifica una parola usando il codice di Hamming
    Argomenti: m (bit di parità), parolaDati (dati da codificare)
    crea parola iniziale e inserisce dati nelle posizioni corrette
    coordina inserimento dati e calcolo bit di parità -}
codificaHamming :: Int -> ParolaBinaria -> ParolaBinaria
codificaHamming m parolaDati =
    let (n, _) = calcolaParametriHamming m
        -- Inizializza una parola di tutti zeri
        parolaIniziale = replicate n 0
        indiciNonParita = filter (\j -> not (potenzaDiDue (j+1)))
            [0..n-1]
        parolaConDati = inserisciDati parolaIniziale indiciNonParita
            parolaDati
    in calcolaParita parolaConDati n m

{- calcolaParita: Calcola tutti i bit di parità per una parola
    Argomenti: parola, n (lunghezza), m (numero bit parità)

```

```

Casi base: quando lista [0..m-1] è vuota ritorna parola invariata
Casi generali: per ogni i in [0..m-1] calcola il bit di parità
    i-esimo
    usa foldl' per applicare calcolaEAggiornaParita a ogni posizione
    di parità -}
calcolaParita :: ParolaBinaria -> Int -> Int -> ParolaBinaria
calcolaParita parola n m = foldl' (calcolaEAggiornaParita n) parola
    [0..m-1]

{- calcolaEAggiornaParita: Calcola e aggiorna un singolo bit di
    parità
    Argomenti: n (lunghezza parola), parola, i (indice bit parità)
    identifica bit controllati, calcola XOR e aggiorna posizione -}
calcolaEAggiornaParita :: Int -> ParolaBinaria -> Int ->
    ParolaBinaria
calcolaEAggiornaParita n parola i =
    let posizioneParita = 2^i
        indiceParita = posizioneParita - 1
        -- testBit j i verifica se il bit i-esimo di j è settato
        indiciControllati = [ j-1 | j <- [1..n], testBit j i ]
        -- Esclude il bit de parità stesso dal calcolo -}
        bitDati = map (parola !!) (filter (/= indiceParita)
            indiciControllati)
        -- XOR di tutti i bit controllati
        valoreXOR = foldl' xor 0 bitDati
    in aggiornaElemento parola indiceParita valoreXOR

{- inserisciDati: Inserisce i dati nelle posizioni specificate
    Argomenti: parola iniziale, lista indici, lista dati
    Casi base: quando liste indici e dati sono vuote ritorna parola
        invariata
    Casi generali: per ogni coppia (indice, dato) aggiorna la parola
        usa foldl' per applicare aggiornaElemento a ogni coppia -}
inserisciDati :: [a] -> [Int] -> [a] -> [a]
inserisciDati parola indici dati =
    -- zip associa ogni indice al dato corrispondente
    foldl' (\acc (indice, dato) -> aggiornaElemento acc indice dato)
        parola (zip indici dati)

{- calcolaSindrome: Calcola la sindrome per rilevare errori
    Argomenti: m (bit parità), parola ricevuta
    Casi base: quando lista [0..m-1] è vuota sindrome = 0
    Casi generali: per ogni bit di parità verifica correttezza e
        accumula errori
    usa foldl' per verificare ogni bit di parità e costruire sindrome
        -}
calcolaSindrome :: Int -> ParolaBinaria -> Int
calcolaSindrome m parola =
    let n = length parola
    in foldl' (\sindrome i ->
        let posizioneParita = 2^i
            indiceParita = posizioneParita - 1
            indiciControllati = [ j-1 | j <- [1..n], testBit j i
                ]
            bitDati = map (parola !!) (filter (/= indiceParita)
                indiciControllati)
            valoreAtteso = foldl' xor 0 bitDati

```

```

        valoreParita = parola !! indiceParita
        {- Se la parità non corrisponde, aggiungi la
           posizione alla sindrome -}
    in if valoreParita /= valoreAtteso
        then sindrome + posizioneParita
        else sindrome
    ) 0 [0..m-1]

{- estraiDati: Estrae i bit di dati da una parola codificata
   Argomenti: m (bit parità), parola codificata
   filtra posizioni non-parità e mappa gli elementi -}
estraiDati :: Int -> ParolaBinaria -> ParolaBinaria
estraiDati m parola =
    let n = length parola
        indiciDati = filter (\i -> not (potenzaDiDue (i+1))) [0..n-1]
    in map (parola !!) indiciDati

{- decodificaHamming: Decodifica una parola e corregge un eventuale
   errore
   Argomenti: m (bit parità), parola ricevuta
   calcola sindrome e decide se correggere o meno -}
decodificaHamming :: Int -> ParolaBinaria -> (ParolaBinaria, Maybe
Int)
decodificaHamming m parola =
    let sindrome = calcolaSindrome m parola
    in if sindrome == 0
        then (estraiDati m parola, Nothing)
        else
            let posizioneErrore = sindrome - 1
                parolaCorretta = aggiornaElemento parola
                    posizioneErrore (1 - parola !! posizioneErrore)
            in (estraiDati m parolaCorretta, Just (posizioneErrore +
1))

{- calcolaDistanzaHamming: Calcola la distanza di Hamming tra due
   parole
   Argomenti: parola1, parola2 -}
calcolaDistanzaHamming :: ParolaBinaria -> ParolaBinaria -> Int
calcolaDistanzaHamming parola1 parola2 =
    sum (zipWith (\a b -> if a == b then 0 else 1) parola1 parola2)

```

4.2 Implementazione in Prolog

Con le stesse motivazioni per l'implementazione in Haskell si fa notare della differenza tra codice presentato qui e quello nel file sorgente. Si prega di consultare "CodiceHamming.pl" per avere la miglior identazione e leggibilità. inizia a pagina successiva.

```

/* #####
# Corso di Programmazione Logica e Funzionale #
# Progetto per la sessione autunnale A.A. 2024/2025 #
# di Nasrine Aboufaris, Matricola: 321885 #
# e Alessia Giuseppetti, Matricola: 322984 #
# Anno di corso: Terzo #
#####
Specifica: Programma Prolog per la codifica/decodifica #
di codici di Hamming e calcolo della distanza di Hamming */

/* Predicato principale del programma. Fornisce istruzioni
   all'utente per l'uso del programma. */
main :-
    write('-----Benvenuto-----'),
    nl,
    write('Il seguente programma permette di utilizzare i codici
          di Hamming generici (n,k) per '), nl,
    write('la codifica, decodifica e calcolo della distanza di
          Hamming tra due parole binarie. '), nl,
    write('Si ricorda che la decodifica di Hamming di seguito
          implementata permette solo di '), nl,
    write('Individuare e correggere un solo errore. '), nl,
    write('Come funziona il programma: '), nl,
    write(' 1. Ti verra\' richiesto il numero di bit di
          parita\' m. '), nl,
    write(' 2. Da m verranno calcolati automaticamente n e
          k. '), nl,
    write(' 3. Segui le istruzioni del menu per completare
          l\'operazione desiderata. '), nl,
    write('La parola deve essere inserita come una lista di
          interi, '), nl,
    write('si prega di usare la seguente formattazione: '), nl,
    write('Scrivere la parola tra parentesi quadre e inserire
          una virgola tra un numero e l\'altro. '), nl,
    write('Esempio di parola accettata: [1, 0, 1, 0]. '), nl,
    write('Esempio di parola non accettata 1010. '), nl,
    write('Si prega di inserire il punto finale prima
          dell\'invio e si raccomanda di seguire questa
          formattazione'), nl,
    write('per evitare anomalie durante l\'esecuzione del
          programma. '), nl,
    write('-----'),
    nl,
    menu.

/* Il predicato menu mostra le opzioni disponibili e
   acquisisce tramite tastiera la scelta dell'utente. */
menu :- write('Scegliere l\'operazione che si desidera eseguire: '),
    nl,
    write(' 1. Codifica di Hamming'), nl,
    write(' 2. Decodifica di Hamming'), nl,
    write(' 3. Distanza di Hamming'), nl,
    write(' 4. Esci'), nl,
    read(Opzione),
    esegui_opzione(Opzione).

/* Il predicato esegui_opzione(1) esegue la codifica di Hamming.
   Chiede il numero di bit di parità M,

```



```

        calcola il parametro k, valida la parola input P di lunghezza k
        ed esegue la codifica.
        - Il primo argomento è l'opzione 1 della scelta dell'utente. */
esegui_opzione(1) :- parita(M),
                    parametri_hamming(M, K, _N),
                    acquisisci_parola(Parola, K),
                    write('Esecuzione codifica...'), nl,
                    codifica_parola(Parola, M),
                    menu.

/* Il predicato esegui_opzione(2) esegue la Decodifica di Hamming.
   Chiede il numero di bit di parità M,
   calcola il parametro n, valida il codice ricevuto di lunghezza n
   ed esegue la decodifica.
   - Il primo argomento è l'opzione 2 della scelta dell'utente. */
esegui_opzione(2) :- parita(M),
                    parametri_hamming(M, _K, N),
                    acquisisci_parola(Parola, N),
                    write('Esecuzione decodifica...'), nl,
                    decodifica_parola(Parola, M),
                    menu.

/* Il predicato esegui_opzione(3) calcola la distanza di Hamming tra
   due parole.
   Acquisisce due parole binarie, ne verifica la validità, controlla
   che abbiano la stessa lunghezza
   e calcola la distanza.
   - Il primo argomento è l'opzione 3 scelta dall'utente. */
esegui_opzione(3) :- write('Inserire la prima parola: '), nl,
                    acquisisci_parola_distanza(T1),
                    write('Inserire la seconda parola: '), nl,
                    acquisisci_parola_distanza(T2),
                    confronto_lunghezza(T1, T2),
                    calcolo_distanza(T1, T2),
                    menu.

/* Il predicato esegui_opzione(4) si occupa di terminare il
   programma.
   Termina l'esecuzione del programma dopo aver stampato un
   messaggio di saluto.
   - Il primo argomento è l'opzione 4 scelta dall'utente. */
esegui_opzione(4) :- write('Grazie per aver utilizzato il programma.
   Arrivederci!'), nl,
                    halt.    % Termina il programma

/* Il predicato esegui_opzione(_) gestisce opzioni non valide.
   Visualizza un messaggio di errore e torna al menu principale.
   - Il primo argomento è un'opzione non compresa tra 1 e 4. */
esegui_opzione(_) :- write('Opzione non valida. Riprovare.'), nl,
                    menu.

/* Il predicato parita(M) acquisisce da tastiera il numero di bit di
   parità scelto
   dall'utente e ne verifica la validità, assicurandosi che il numero
   sia maggiore o uguale a 2. */
parita(M) :- nl, write('Il bit di parita\' deve essere maggiore o
   uguale a 2. '), nl,
            write('Inserire il bit di parita\' scelto:'), nl,

```

```

        % lettura dell'input
        (catch(read(M), _, fail),
        % validazione dell'input
        integer(M),
        M >= 2
        ->
            true
        ;
            nl, write('Input non valido! Deve essere un intero
                    >= 2. '), nl,
            parita(M)
        ).

/* Il predicato parametri_hamming(M, K, N) calcola i parametri k(bit
di dati) ed n(bit totali)
per il codice di Hamming.
- Il primo argomento rappresenta il numero di bit di parità.
- Il secondo argomento rappresenta il numero di bit di dati.
- Il terzo argomento rappresenta il numero totale di bit.
Esempio: parametri_hamming(3, K, N) calcola K=4 e N=7. */
parametri_hamming(M,K,N):- K is 2^M - 1 - M,
                           N is 2^M -1.

/* Il predicato acquisisci_parola acquisisce una parola binaria
dall'input
e la valida rispetto alla lunghezza specificata.
- Il primo argomento è la parola validata in output.
- Il secondo argomento è la lunghezza attesa della parola. */
acquisisci_parola(Parola, Lunghezza) :- write('Inserire la parola di
lunghezza '),
                                        write(Lunghezza),
                                        write(': '),
                                        (catch(read(Input), _, fail)
                                        -> processa_input(Input,
                                                Lunghezza, Parola)
                                        ;   nl, write('Errore di
                                                acquisizione!'), nl,
                                                acquisisci_parola(Parola,
                                                Lunghezza)
                                        ).

/* Il predicato processa_input gestisce la validazione dell'input e
la ricorsione
in caso di errore. Se l'input è valido, unifica Parola con Input;
altrimenti,
stampa un messaggio di errore e richiede nuovo input.
- Il primo argomento è l'input letto da tastiera.
- Il secondo argomento è la lunghezza attesa.
- Il terzo argomento è l'output validato. */
processa_input(Input, Lunghezza, Parola) :-
    (valida_parola_binaria(Input, Lunghezza)
    -> Parola = Input
    ;   write('Parola non
            valida. Deve essere
            lunga '),
            write(Lunghezza),
            write(' e contenere
                    solo 0 e 1. '), nl,

```

```

                                acquisisci_parola(Parola,
                                                Lunghezza)
                                ).

/* Il predicato valida_parola_binaria verifica che una lista sia
   valida per il codice di Hamming, controllando che sia
   una lista, che abbia che abbia lunghezza Lunghezza e che tutti
   gli elementi siano bit validi.
   - il primo argomento è la lista da verificare
   - il secondo argomento è la lunghezza della parola attesa. */
valida_parola_binaria(Parola, Lunghezza) :- is_list(Parola),
                                            length(Parola,
                                                Lunghezza),
                                            maplist(bit_valido,
                                                Parola).

/* Il fatto bit_valido definisce i bit validi per il codice di
   Hamming.
   Un bit è valido se è 0 o 1.
   - L'unico argomento rappresenta il bit da verificare. */
bit_valido(0).
bit_valido(1).

/* Il predicato acquisisci_parola_distanza acquisisce una parola
   binaria generica dall'input
   e la valida.
   - L'unico argomento è la parola validata in output. */
acquisisci_parola_distanza(Parola) :- (catch(read(Input), _, fail),
                                       processa_input_distanza(Input,
                                           Parola)
                                       ; nl, write('Errore di
                                           acquisizione!'), nl,
                                       acquisisci_parola_distanza(Parola)
                                       ).

/* Il predicato processa_input_distanza gestisce la validazione
   dell'input e la ricorsione
   in caso di errore. Se l'input è valido, unifica Parola con Input;
   altrimenti,
   stampa un messaggio di errore e richiede nuovo input.
   - Il primo argomento Input è l'input letto da tastiera.
   - Il secondo argomento Parola è l'output validado. */
processa_input_distanza(Input, Parola) :-
    (validazione_parola_distanza(Input),
        Parola = Input
    ; write('Parola non
        valida. Deve essere una
        lista contenente solo 0
        e 1.'), nl,
        acquisisci_parola_distanza(Parola)
    ).

/* Il predicato validazione_parola_generica verifica che l'input sia
   una lista
   composta esclusivamente da valori binari (0 e 1).
   - L'unico argomento rappresenta la parola da validare. */

```

```

validazione_parola_distanza(Parola) :- is_list(Parola),
                                         maplist(bit_valido, Parola).

/* Il predicato confronto_lunghezza verifica che due liste abbiano
   la stessa lunghezza,
   condizione necessaria per il calcolo della distanza di Hamming.
   - Il primo argomento rappresenta la prima lista da confrontare.
   - Il secondo argomento rappresenta la seconda lista da
     confrontare. */
confronto_lunghezza(T1, T2) :- length(T1, L1),
                                length(T2, L2),
                                (L1 == L2
                                -> true
                                ; write('Le due parole devono avere
                                    la stessa lunghezza. '), nl,
                                  write('Prima parola lunghezza: '),
                                  write(L1), nl,
                                  write('Seconda parola lunghezza:
                                    '), write(L2), nl,
                                  write('Inserire nuovamente le due
                                    parole. '), nl,
                                  esegui_opzione(3)
                                ).

/* Il predicato posizione_parita calcola, dato un indice, se la
   posizione è potenza di due o meno.
   - L'unico argomento indica la posizione che gli viene passata. */
posizione_parita(Posizione) :- Posizione > 0,
                                (Posizione /\ (Posizione - 1)) == 0.

/* Il predicato inserisci_bit inserisce bit di dati nelle posizioni
   non di parità di un codice Hamming,
   lasciando dei segnaposti per i bit di parità.
   - Il primo argomento rappresenta la posizione corrente nel
     codice.
   - Il secondo argomento è la lista di bit di dati da inserire.
   - Il terzo argomento è il codice risultante con i bit di dati
     nelle posizioni appropriate. */
inserisci_bit(_, [], []) :- !.
inserisci_bit(Pos, [B|Coda], [B|Resto]) :- \+ posizione_parita(Pos),
                                             PosSucc is Pos + 1,
                                             inserisci_bit(PosSucc,
                                                             Coda, Resto).
inserisci_bit(Pos, BitDati, [_|Resto]) :- posizione_parita(Pos),
                                             PosSucc is Pos + 1,
                                             inserisci_bit(PosSucc,
                                                             BitDati, Resto).

/* Il predicato calcolo_parita calcola il valore dei bit di parità
   per una specifica posizione P
   in una lista di bit, utilizzando l'operazione XOR tra i bit nelle
   posizioni appropriate.
   - Il primo argomento è la lista di bit su cui operare.
   - Il secondo argomento indica la posizione del bit di parità da
     calcolare.
   - Il terzo argomento è il valore calcolato del bit di parità. */
calcolo_parita(Lista, Pos, Valore_P) :- calcolo_parita(Lista, Pos,
1, 0, Valore_P).

```

```

/* Il predicato calcolo_parita implementa il calcolo ricorsivo del
   bit di parità.
   - Il primo argomento è la lista di bit rimanente da processare.
   - Il secondo argomento è la posizione del bit di parità.
   - Il terzo argomento è la posizione corrente nella lista.
   - Il quarto argomento è l'accumulatore per il calcolo XOR.
   - Il quinto argomento è il valore finale del bit di parità. */
calcolo_parita([], _, _, Acc, Acc).
calcolo_parita([Bit|Resto], Pos, Indice, Acc, Valore_P) :- ((Indice
/\ Pos) > 0 ->
                                                    xor_bit(Acc,
                                                    Bit,
                                                    NuovoAcc)
;
                                                    NuovoAcc
                                                    = Acc
),
ProssimoIndice
is
Indice
+ 1,
calcolo_parita(Resto,
Pos,
ProssimoIndice,
NuovoAcc,
Valore_P).

/* Fatto xor_bit implementa l'operazione XOR tra due bit.
   - Il primo argomento è il primo bit.
   - Il secondo argomento è il secondo bit.
   - Il terzo argomento è il risultato dell'operazione XOR. */
xor_bit(0, 0, 0).
xor_bit(0, 1, 1).
xor_bit(1, 0, 1).
xor_bit(1, 1, 0).

/* Il predicato sostituisci_elemento sostituisce l'elemento in una
   specifica posizione di una lista.
   - Il primo argomento è la lista originale.
   - Il secondo argomento è la posizione dell'elemento da sostituire.
   - Il terzo argomento è il nuovo valore da inserire.
   - Il quarto argomento è la lista risultante dopo la sostituzione.
   */
sostituisci_elemento([_|T], 1, X, [X|T]).
sostituisci_elemento([H|T], Pos, X, [H|T1]) :- Pos > 1,
                                                    Pos1 is Pos - 1,
                                                    sostituisci_elemento(T,
                                                    Pos1, X, T1).

/* Il predicato calcola_tutte_parita calcola tutti i bit di parità
   per una lista di posizioni
   e li inserisce nel codice.
   - Il primo argomento è il codice temporaneo senza bit di parità.
   - Il secondo argomento è la lista delle posizioni dei bit di
   parità.

```

```

- Il terzo argomento è il codice completo con tutti i bit di
  parità calcolati. */

calcola_tutte_parita(List, [], List).
calcola_tutte_parita(List, [P|Resto], Risultato) :-
    calcolo_parita(List, P, PV),
    sostituisci_elemento(List,
        P, PV,
        NuovaLista),
    calcola_tutte_parita(NuovaLista,
        Resto,
        Risultato).

/* Il predicato parita_list genera la lista delle posizioni dei bit
   di parità per un dato numero M.
   Le posizioni sono le potenze di 2 fino a 2^(M-1).
   - Il primo argomento è il numero di bit di parità.
   - Il secondo argomento è la lista delle posizioni dei bit di
     parità. */
parita_list(M, List) :- M1 is M - 1,
    findall(P, (between(0, M1, Exp), P is
        2^Exp), List).

/* Il predicato codifica_parola implementa la codifica di Hamming
   per una parola di dati.
   - Il primo argomento è la parola di dati da codificare.
   - Il secondo argomento è il numero di bit di parità.
   Il predicato inserisce i bit di dati nelle posizioni appropriate,
   calcola i bit di parità e produce il codice di Hamming finale. */
codifica_parola(Parola, M) :- % Calcola le posizioni dei bit di
    parità
    parita_list(M, PosizioniParita),
    % Inserisce i bit di dati nelle
    % posizioni appropriate
    inserisci_bit(1, Parola, CodiceTemp),
    % Calcola i valori dei bit di parità
    % e li inserisce
    calcola_tutte_parita(CodiceTemp,
        PosizioniParita, CodiceHamming),
    write('Codice Hamming: '),
    write(CodiceHamming), nl.

/* Il predicato calcola_sindrome calcola la sindrome per un codice
   Hamming ricevuto.
   La sindrome rappresenta la somma delle posizioni dei bit di
   parità che risultano errati.
   Se la sindrome è 0, non ci sono errori, in caso contrario indica
   la posizione dell'errore.
   - Il primo argomento è il codice di Hamming da analizzare.
   - Il secondo argomento è la lista delle posizioni dei bit di
     parità.
   - Il terzo argomento è il valore calcolato della sindrome. */
calcola_sindrome(_, [], 0).
calcola_sindrome(Parola, [P|Resto], Sindrome) :-
    calcolo_parita(Parola, P, Valore_P),

```

```

        calcola_sindrome(Parola,
            Resto,
            SindromeParziale),
        (Valore_P := 1 ->
            Sindrome is
                SindromeParziale
                + P
        ; Sindrome =
            SindromeParziale).

/* Il predicato elemento_in restituisce l'elemento in una specifica
   posizione di una lista
   utilizzando la numerazione basata su 1 (il primo elemento è in
   posizione 1).
   - Il primo argomento è la lista in cui cercare.
   - Il secondo argomento è la posizione dell'elemento da estrarre.
   - Il terzo argomento è l'elemento trovato nella posizione
     specificata. */
elemento_in([X|_], 1, X).
elemento_in([_|T], Pos, X) :- Pos > 1,
    Pos1 is Pos - 1,
    elemento_in(T, Pos1, X).

/* Il predicato correggi_errore corregge un singolo errore bit in un
   codice di Hamming
   invertendo il valore del bit nella posizione specificata (0
   diventa 1, 1 diventa 0).
   - Il primo argomento è il codice originale contenente l'errore.
   - Il secondo argomento è la posizione del bit errato.
   - Il terzo argomento è il codice con l'errore corretto. */
correggi_errore(Lista, Pos, ListaCorretta) :- elemento_in(Lista,
    Pos, Bit),
    (Bit == 0 -> NuovoBit
        = 1 ; NuovoBit = 0),
    sostituisci_elemento(Lista,
        Pos, NuovoBit,
        ListaCorretta).

/* Il predicato estrai_dati estrae i bit di dati da un codice di
   Hamming,
   rimuovendo i bit di parità che si trovano nelle posizioni che
   sono potenze di 2.
   - Il primo argomento è il codice Hamming completo.
   - Il secondo argomento è la lista contenente solo i bit di dati.
   */
estrai_dati(Codice, Dati) :- estrazione_dati(Codice, 1, Dati).

/* Predicato ricorsivo per estrai_dati che effettua l'estrazione
   effettiva
   dei bit di dati, saltando i bit di parità nelle posizioni che
   sono potenze di 2.
   - Il primo argomento è la porzione rimanente del codice da
     processare.
   - Il secondo argomento è la posizione corrente nel codice
     (contatore).
   - Il terzo argomento è la lista accumulata dei bit di dati
     estratti. */

```

```

estrazione_dati([], _, []).
estrazione_dati([Bit|Resto], Pos, [Bit|DatiResto]) :- \+
    posizione_parita(Pos),
    !,
    PosSucc is Pos
    + 1,
    estrazione_dati(Resto,
        PosSucc,
        DatiResto).
estrazione_dati([_|Resto], Pos, Dati) :- posizione_parita(Pos),
    !,
    PosSucc is Pos + 1,
estrazione_dati(Resto, PosSucc, Dati).

/* Il predicato decodifica_parola implementa la decodifica completa
di un codice di Hamming.
Calcola la sindrome del codice ricevuto: se è zero non ci sono
errori, altrimenti
corregge l'errore nella posizione indicata e estrae i bit di dati.
- Il primo argomento CodiceRicevuto è il codice Hamming da
decodificare.
- Il secondo argomento M è il numero di bit di parità utilizzati.
*/
decodifica_parola(CodiceRicevuto, M) :- parita_list(M,
    PosizioniParita),
    calcola_sindrome(CodiceRicevuto,
        PosizioniParita,
        Sindrome),
    (Sindrome == 0 ->
        write('Nessun errore
            trovato.'), nl,
        estrai_dati(CodiceRicevuto,
            Dati)
    ; write('Errore trovato in
        posizione: '),
        write(Sindrome), nl,
        correggi_errore(CodiceRicevuto,
            Sindrome,
            CodiceCorretto),
        estrai_dati(CodiceCorretto,
            Dati)
    ),
    write('Parola decodificata:
        '), write(Dati), nl.

/* Predicato che esegue il calcolo di Hamming in maniera ricorsiva.
- il primo argomento indica la prima lista inserita;
- il secondo argomento indica la seconda lista inserita;
- il terzo argomento è la distanza di Hamming risultante.
Il calcolo della distanza è stato fatto in maniera ricorsiva:*/

calcolo_distanza([], [], 0).
calcolo_distanza([H1|T1], [H2|T2], D) :- calcolo_distanza(T1, T2,
    D1),
    ( H1 == H2
    -> D = D1
    ; D is D1 + 1
    ).

```



```
calcolo_distanza(T1, T2) :- calcolo_distanza(T1, T2, D),  
                             write('Distanza di Hamming: '),  
                             write(D), nl.
```

5 Testing

5.1 Testing del programma Haskell

Test nr. 1 - Errore nella scelta del menu

```
"-----Benvenuto-----"
"Il seguente programma permette di utilizzare i codici di Hamming generici (n,k)"
"per la codifica, decodifica e calcolo della distanza di Hamming tra due parole"
"binarie."
"Come funziona il programma:"
"Potrai scegliere dal menu una delle tre operazioni da eseguire, per la codifica"
"e decodifica di Hamming, verra' chiesto di inserire m, ovvero il numero di bit"
"di parita', grazie ad m verranno calcolati n e k, e si potra' inserire la"
"parola binaria. Esempio di parola accetta '1011'"
"Notare che per la decodifica questo algoritmo corregge solo errori singoli."
"Con errori multipli il risultato potrebbe essere errato."
"Il programma finisce solo nel momento in cui si sceglie l'opzione esci dal menu"
"-----"
```

Scegli operazione:

- 1 - Codifica di Hamming
- 2 - Decodifica di Hamming
- 3 - Distanza di Hamming
- 4 - Esci

scelta: 5

Scelta non valida. Riprova.

Test nr. 2 - Errore m minore di 2

Scegli operazione:

- 1 - Codifica di Hamming
- 2 - Decodifica di Hamming
- 3 - Distanza di Hamming
- 4 - Esci

scelta: 1

Codifica di Hamming

Inserisci m ($m \geq 2$):

1

Errore: m deve essere un intero maggiore uguale a 2

Test nr. 3 - Errore nella lunghezza della parola da codificare

Scegli operazione:

- 1 - Codifica di Hamming
- 2 - Decodifica di Hamming
- 3 - Distanza di Hamming
- 4 - Esci

scelta: 1
Codifica di Hamming
Inserisci m ($m \geq 2$):
3
Parametri calcolati: $n = 7$, $k = 4$
Inserisci parola binaria di lunghezza 4:
10000
Errore: lunghezza non valida o caratteri non binari

Test nr. 4 - Codifica con $m = 3$

Scegli operazione:
1 - Codifica di Hamming
2 - Decodifica di Hamming
3 - Distanza di Hamming
4 - Esci

scelta: 1
Codifica di Hamming
Inserisci m ($m \geq 2$):
3
Parametri calcolati: $n = 7$, $k = 4$
Inserisci parola binaria di lunghezza 4:
1010
La parola codificata è: 1011010

Test nr. 5 - Errore nella lunghezza della parola da decodificare

Scegli operazione:
1 - Codifica di Hamming
2 - Decodifica di Hamming
3 - Distanza di Hamming
4 - Esci

scelta: 2
Decodifica di Hamming
Inserisci m ($m \geq 2$):
3
Parametri calcolati: $n = 7$, $k = 4$
Inserisci parola binaria di lunghezza 7:
101101
Errore: lunghezza non valida o caratteri non binari

Test nr. 6 - Errore inserimento di una parola non binaria nella decodifica

Scegli operazione:
1 - Codifica di Hamming
2 - Decodifica di Hamming
3 - Distanza di Hamming

4 - Esci

scelta: 2

Decodifica di Hamming

Inserisci m ($m \geq 2$):

3

Parametri calcolati: $n = 7$, $k = 4$

Inserisci parola binaria di lunghezza 7:

1011013

Errore: lunghezza non valida o caratteri non binari

Test nr. 7 - Decodifica senza errori

Scelta: 2

Inserisci m ($m \geq 2$):

3

Parametri calcolati: $n = 7$, $k = 4$

Inserisci parola binaria di lunghezza 7:

1011010

Nessun errore rilevato. La parola decodificata è: 1010

Test nr. 8 - Decodifica con singolo errore

Scelta: 2

Decodifica di Hamming

Inserisci m ($m \geq 2$):

3

Parametri calcolati: $n = 7$, $k = 4$

Inserisci parola binaria di lunghezza 7:

1110011

Corretto un errore nella posizione 1. La parola decodificata è: 1011

Test nr. 9 - Errore nella lunghezza delle due parole nel calcolo distanza di Hamming

Scelta: 3

Distanza di Hamming

Inserisci prima parola binaria:

10101

Inserisci seconda parola binaria (stessa lunghezza):

001100

Errore: le parole devono essere binarie e di uguale lunghezza

Test nr. 10 - Calcolo distanza di Hamming

Scelta: 3

Distanza di Hamming

Inserisci prima parola binaria:

10101

Inserisci seconda parola binaria (stessa lunghezza):

10001

La distanza di Hamming tra le due parole è: 1

5.2 Testing del programma Prolog

Test nr. 1 - Errore nella scelta del menu

-----Benvenuto-----

Il seguente programma permette di utilizzare i codici di Hamming generici (n,k) per la codifica, decodifica e calcolo della distanza di Hamming tra due parole binarie. Si ricorda che la decodifica di Hamming di seguito implementata permette solo di Individuare e correggere un solo errore.

Come funziona il programma:

1. Ti verra' richiesto il numero di bit di parita' m.
2. Da m verranno calcolati automaticamente n e k.
3. Segui le istruzioni del menu per completare l'operazione desiderata.

La parola deve essere inserita come una lista di interi, si prega di usare la seguente formattazione:

Scrivere la parola tra parentesi quadre e inserire una virgola tra un numero e l'altro.

Esempio di parola accettata: [1, 0, 1, 0].

Esempio di parola non accettata 1010.

Si prega di inserire il punto finale prima dell'invio e si raccomanda di seguire questa formattazione

per evitare anomalie durante l'esecuzione del programma.

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
 2. Decodifica di Hamming
 3. Distanza di Hamming
 4. Esci
 - 5.
- Opzione non valida. Riprovare.

Test nr. 2 - Errore m minore di 2

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
2. Decodifica di Hamming
3. Distanza di Hamming
4. Esci
- 1.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

0.

Input non valido! Deve essere un intero ≥ 2 .

Test nr. 3 - Errore nella lunghezza della parola da codificare

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
2. Decodifica di Hamming
3. Distanza di Hamming
4. Esci

1.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

3.

Inserire la parola di lunghezza 4: [1,0,1,1,1,1].

Parola non valida. Deve essere lunga 4 e contenere solo 0 e 1.

Inserire la parola di lunghezza 4:

Test nr. 4 - Codifica con $m = 3$

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
2. Decodifica di Hamming
3. Distanza di Hamming
4. Esci

1.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

3.

Inserire la parola di lunghezza 4: [1,0,1,0].

Esecuzione codifica...

Codice Hamming: [1,0,1,1,0,1,0]

Test nr. 5 - Errore nella lunghezza della parola da decodificare

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
2. Decodifica di Hamming
3. Distanza di Hamming
4. Esci

2.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

5.

Inserire la parola di lunghezza 31: [0,1,1,1,].

Errore di acquisizione!

Inserire la parola di lunghezza 31:

Test nr. 6 - Errore inserimento di una parola non binaria nella decodifica

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
 2. Decodifica di Hamming
 3. Distanza di Hamming
 4. Esci
- 2.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

4.

Inserire la parola di lunghezza 15: [1,0,1,1,1,3,4,5,0,0,1,1,0,1,0].

Parola non valida. Deve essere lunga 15 e contenere solo 0 e 1.

Test nr. 7 - Decodifica senza errori

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
 2. Decodifica di Hamming
 3. Distanza di Hamming
 4. Esci
- 2.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

3.

Inserire la parola di lunghezza 7: [1,0,1,1,0,1,0].

Esecuzione decodifica...

Nessun errore trovato.

Parola decodificata: [1,0,1,0]

Test nr. 8 - Decodifica con singolo errore

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
 2. Decodifica di Hamming
 3. Distanza di Hamming
 4. Esci
- 2.

Il bit di parita' deve essere maggiore o uguale a 2.

Inserire il bit di parita' scelto:

3.

Inserire la parola di lunghezza 7: [1,1,1,1,0,1,0].

Esecuzione decodifica...

Errore trovato in posizione: 2

Parola decodificata: [1,0,1,0]

Test nr. 9 - Errore nella lunghezza delle due parole nel calcolo distanza di Hamming

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
2. Decodifica di Hamming
3. Distanza di Hamming
4. Esci

3.

Inserire la prima parola:

[1,1,1,0,1,0].

Inserire la seconda parola:

[1,1,1,0].

Le due parole devono avere la stessa lunghezza.

Prima parola lunghezza: 6

Seconda parola lunghezza: 4

Inserire nuovamente le due parole.

Inserire la prima parola:

Test nr. 10 - Calcolo distanza di Hamming

Scegliere l'operazione che si desidera eseguire:

1. Codifica di Hamming
2. Decodifica di Hamming
3. Distanza di Hamming
4. Esci

3.

Inserire la prima parola:

[1,0,1,1,0].

Inserire la seconda parola:

[1,1,0,1,0].

Distanza di Hamming: 2